

SalesBench: A Long-Horizon, Agent-to-Agent Benchmark for Tool-Use Sales Agents

Anonymous authors

Paper under double-blind review

Abstract

Most agentic benchmarks score a single trajectory against a static rubric or a fixed verifier. Real economically valuable agent tasks are instead *long-horizon* and *agent-to-agent*: an agent operates a stateful workflow under a hard resource budget while interacting with another adaptive agent whose decisions it cannot control. We introduce SALESBENCH, a benchmark that casts insurance sales as such a task. A seller agent is dropped into a deterministic simulator with ~ 100 procedurally generated leads and a fixed time budget, and must maximize converted monthly recurring revenue by searching a CRM, quoting plans, placing calls, and closing deals—while a separate buyer language model, instantiated with one of ten personality archetypes, decides whether to accept, reject, or hang up. The environment is a pure deterministic state machine: given a seed, every lead, price, and time cost is reproducible, so variance comes only from the two agents, which is what makes multi-sample evaluation meaningful. SALESBENCH cleanly separates seller mistakes (which are penalized) from buyer-model failures (which are not), and ships a buyer-prompt ablation axis (DEFAULT, SKEPTICAL, IMPULSIVE, ANALYTICAL) to probe whether an agent has learned to sell or merely overfit one counterparty. As a first demonstration of what the benchmark measures, we run a four-stage difficulty curriculum that trains a 2B-parameter model with online reinforcement learning. The trained agent raises per-lead conversion from 0.3% to 18.5% at 100 leads (composite reward $-0.04 \rightarrow 0.19$), and the advantage over the untrained base *widens* with horizon. It holds 8–20% per-lead conversion across all four buyer-prompt variants—including the three it never trained against—and *tops a leaderboard of frontier API models by roughly 3 $\times$* , evidence that what the benchmark measures is an operational policy that does not come for free with scale. We release the environment, curriculum, and evaluation harness.

1 Introduction

Language-model agents are increasingly deployed on tasks where success is not a single correct answer but the cumulative outcome of many decisions made under a budget. A coding agent navigates a repository across dozens of tool calls; a research agent reads, synthesizes, and revises; a customer-facing agent negotiates across a multi-turn conversation. Yet the benchmarks we use to evaluate agents have largely kept the shape of question answering: a prompt, a trajectory, and a verifier that scores the trajectory against a fixed target (Liu et al., 2024; Mialon et al., 2024). This shape is convenient because it is deterministic and gradeable, but it omits two features that characterize the most economically valuable agentic work.

The first is **horizon under a hard budget**. A real agent does not get to take an arbitrary number of actions; it has a finite amount of time, money, or context, and good behavior means *allocating* that budget—deciding which leads to pursue, when to stop talking and act, and when to cut losses. The second is the **agent-to-agent** structure. The environment is not a passive grader but another adaptive agent with its own goals, information, and personality, whose responses the protagonist cannot dictate and must instead influence.

45 Sales, negotiation, tutoring, and customer support all have this structure, and all are domains
46 where current models are being deployed with little principled measurement.

47 Existing benchmarks tend to supply one feature but not both, and to lean on a grading
48 mode that production agents never enjoy. Long-horizon business simulators are stateful and
49 budgeted but pair the agent with *scripted* customers; tool-agent-user benchmarks supply
50 an adaptive language-model counterparty but in *short* episodes; and many agentic suites
51 score either narrow task completion or a judge model’s verdict, when a deployed sales
52 or support agent gets neither a gold trajectory nor a judge. SALESBENCH is built to sit at
53 the intersection: long-horizon, stateful, budgeted, driven by an adaptive language-model
54 counterparty, and graded by a verifiable outcome read directly from environment state—did
55 a deal close, and at what premium.

56 We introduce SALESBENCH, a benchmark that captures both features in a single, repro-
57 ducible task: simulated insurance sales. The setup is deliberately mundane and econom-
58 ically legible. A *seller* agent is given a pool of ~ 100 prospective buyers (“leads”), each
59 with demographics, latent need, trust, price sensitivity, and a monthly budget, and a hard
60 time budget of, e.g., 50 simulated working hours. Each action—searching the customer
61 relationship management (CRM) system, pulling a quote, starting a call, proposing an
62 offer—costs simulated minutes. The seller’s objective is to maximize *converted monthly*
63 *recurring revenue* (MRR): the sum of premiums on deals it actually closes. Closing requires a
64 separate *buyer* language model to accept the offer, and the buyer is instantiated per-lead
65 with one of ten personality archetypes (Analytical, Skeptic, Budget Hawk, Protector, and
66 so on) and a “temperature” from cold to hot. The buyer decides, in character, whether to
67 accept, reject, or hang up.

68 This design yields three properties we argue a long-horizon agent-to-agent benchmark
69 should have.

70 **Reproducible world, stochastic agents.** The simulator (§3) is a pure deterministic state
71 machine with zero internal model calls. Given a seed, the lead population, the product
72 prices, the per-action time costs, and every state transition are bit-for-bit reproducible. The
73 *only* sources of variance in an episode are the seller and buyer models. This is the property
74 that makes running k rollouts per scenario informative: differences across rollouts are
75 attributable to the policies, not to a shifting world.

76 **Failure attribution.** In an agent-to-agent task the protagonist can fail for two very different
77 reasons—it made a bad decision, or its counterparty (here, a sometimes-flaky language
78 model) misbehaved. SALESBENCH separates these by construction (§3.6). Seller errors
79 (proposing without a quote, calling a do-not-call lead, malformed arguments) are deter-
80 ministic and *are* penalized; buyer-model failures (timeouts, malformed JSON) are isolated
81 and resolve to a benign rejection that *never* penalizes the seller. Without this separation, an
82 agent’s score would be contaminated by the reliability of the buyer’s serving infrastructure.

83 **A built-in generalization axis.** Overfitting to one counterparty is the central risk in agent-
84 to-agent training. SALESBENCH ships four buyer-prompt variants that share identical buyer
85 information but differ in decision style, so an agent can be trained against one and evaluated
86 against the other three. An agent that has learned “how to sell” should transfer; an agent
87 that has learned “how to please the DEFAULT buyer” should not.

88 To show the benchmark is not merely a static leaderboard but a trainable signal, we run a
89 four-stage difficulty curriculum (2, 4, 8, then 20 leads) that trains a 2B-parameter model with
90 online reinforcement learning against the DEFAULT buyer, then evaluate it at full scale (50
91 and 100 leads) against all four buyers (§4). The trained agent improves over the untrained
92 base by a mean conversion lift of $21\times$ at 50 leads and $50\times$ at 100 leads; the gap *grows*
93 with horizon, because the untrained model collapses as the world gets longer while the
94 trained one degrades gracefully. It sustains 8–20% per-lead conversion across all four buyer
95 variants—including the three it never trained against—which we read as evidence of a
96 transferable policy rather than counterparty overfitting.

97 Our contributions are: (1) SALESBENCH, a reproducible long-horizon agent-to-agent bench-
98 mark for tool-use sales agents, with explicit seller/buyer failure attribution and a buyer-

variant generalization axis; (2) a deterministic simulator, ten-archetype buyer model, revenue-first reward, and difficulty curriculum, all released open-source; (3) a demonstration that a small model trained on the benchmark learns a policy that generalizes across unseen counterparties and whose advantage widens with horizon; and (4) a frontier-model leaderboard showing that the benchmark’s ranking is not a function of scale—a 120B model scores at the noise floor while the curriculum-trained 2B model tops every untrained API model evaluated, by roughly $3\times$.

2 Related work

Agentic and tool-use benchmarks. A growing body of work evaluates language models as agents that call tools and act over multiple turns. AgentBench (Liu et al., 2024) and GAIA (Mialon et al., 2024) aggregate diverse agentic tasks; ToolLLM (Qin et al., 2024) and Gorilla (Patil et al., 2024) focus on API/tool invocation; WebArena and SWE-bench (Zhou et al., 2024; Jimenez et al., 2024) study realistic web and software-engineering environments. These benchmarks are valuable but are predominantly graded against a static target—a gold trajectory, a passing test suite, a reachable goal state. SALESBENCH differs in that the grader is itself an adaptive agent and the objective is a continuous resource-allocation outcome (revenue under a time budget) rather than task completion.

Agent-to-agent and tool-agent-user simulation. The closest line of work simulates a second agent as part of the environment. τ -bench and τ^2 -bench (Yao et al., 2024; Barres et al., 2025) introduce a user-simulator language model that the agent must serve while respecting domain policies, and emphasize the three-party structure (agent, simulated user, environment) that we adopt. SALESBENCH shares this orchestration pattern but targets a different regime: the counterparty is adversarially uncooperative by design (it can reject and hang up), the objective is competitive revenue maximization rather than policy-compliant task completion, and the horizon is an order of magnitude longer (hundreds of tool calls across ~ 100 independent counterparties under a single shared budget). Negotiation dialogue (Lewis et al., 2017) and persuasion (Wang et al., 2019) study agent-to-agent influence in a single conversation; SALESBENCH embeds many such conversations inside a portfolio-level budgeting problem.

Long-horizon, budgeted agents. Vending-Bench (Backlund & Petersson, 2025) evaluates agents that run a simulated business over long horizons with persistent state and a real budget, and is the closest benchmark to ours in operational shape. The two existing strands are, however, complementary rather than overlapping: Vending-Bench is long-horizon and stateful but its customers are *scripted*, whereas τ -bench has an adaptive language-model counterparty but its episodes are *short*. SALESBENCH is designed to occupy the intersection—long-horizon *and* stateful *and* driven by an adaptive language-model counterparty *and* resource-constrained *and* graded by a verifiable, state-derived outcome. Crucially, the score comes purely from environment state (did a sale close, at what premium), with no judge model in the scoring loop. Language-model judges are known to be biased and inconsistent (Zheng et al., 2023), and production agents in sales, support, and scheduling get neither a gold trajectory nor a judge; we argue an evaluation of them should rely on neither either.

Reinforcement learning for agents. Reinforcement learning from verifiable rewards has driven recent progress in reasoning and instruction following (Shao et al., 2024; Lambert et al., 2024), and SALESBENCH extends the verifiable-reward setting to a long-horizon, agent-to-agent task. We use Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which is well suited to grouped rollouts from the same scenario, and a curriculum (Bengio et al., 2009) over episode difficulty. Our environment is implemented on the VERIFIERS framework for reinforcement-learning environments (Brown, 2025), whose stateful tool-environment abstraction supports the per-rollout simulator state and hidden tool arguments we require.

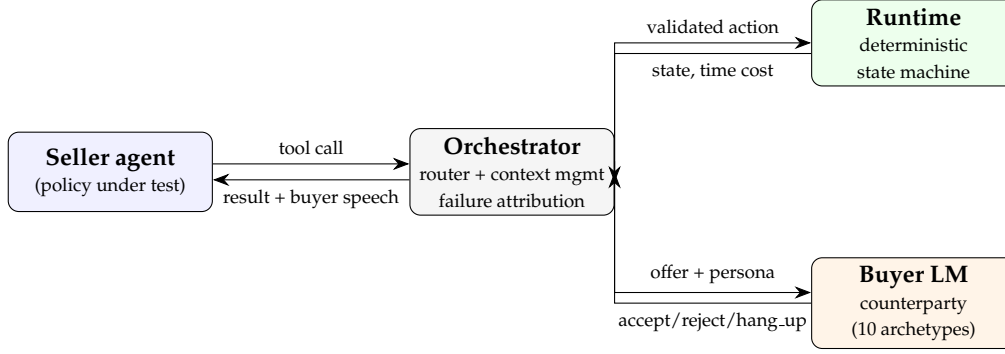


Figure 1: Three-party orchestration. The seller agent (the policy under evaluation) only ever talks to the orchestrator. The orchestrator routes validated actions into a deterministic runtime (which owns all state and time accounting) and routes offers to a buyer language model (the counterparty). Seller errors are attributed to the seller and penalized; buyer-model failures are isolated and resolve to a benign rejection. The seller never sees the runtime or the buyer handle directly.

148 3 The SALESBENCH environment

149 3.1 Task and objective

150 An episode places a seller agent in a sales pipeline defined by a seed. The agent receives
 151 a brief and a set of tools, and acts turn by turn until the episode terminates. The world
 152 consists of: a population of N leads (default $N=100$); a product catalog of four insurance
 153 plan types; a wall clock measured in simulated minutes with budget $B = H \times 60$ (default
 154 $H=50$ hours, $B=3000$ minutes); and a set of per-action time costs. Each lead i has a monthly
 155 budget b_i . The agent earns revenue only by getting the buyer to *accept* an offer, at which
 156 point the accepted premium p_i is added to MRR. The buyer weighs the premium against its
 157 budget when deciding (offers far above budget are typically rejected), but the budget is not
 158 a hard cap—whether an over-budget offer is accepted depends on the lead’s archetype and
 159 temperature, so pricing to budget is a learned skill rather than an enforced constraint. The
 160 episode ends when the time budget is exhausted, all leads are processed (converted, on the
 161 do-not-call list, or out of call attempts), or the agent accrues too many invalid actions.

162 3.2 Three-party orchestration

163 SALESBENCH follows the agent-counterparty-environment decomposition of τ^2 -bench (Bar-
 164 res et al., 2025), separated into three components with disjoint responsibilities (Figure 1).

165 **Runtime (deterministic state machine).** A pure Python state machine owns all episode
 166 state—leads, clock, active call, callbacks, statistics, and termination—and contains *zero*
 167 model calls. It exposes methods for each tool action, validates them, advances the clock, and
 168 records outcomes. Determinism is enforced at construction: a single seeded random number
 169 generator produces the entire lead population, and prices are a closed-form function of
 170 inputs. We verify reproducibility: two runtimes built from the same seed produce identical
 171 leads, and the maximum achievable MRR equals the exact sum of lead budgets.

172 **Buyer policy (counterparty model).** A separate language model simulates the buyer. On
 173 each proposed offer it is prompted with the lead’s full persona and the call transcript so
 174 far and returns a structured decision in {accept, reject, hang-up} with a natural-language
 175 reason; during conversation it replies in character. The buyer lives entirely outside the
 176 runtime, which never sees a model call.

177 **Orchestrator (router).** A stateful tool-environment routes the seller’s tool calls into the
 178 runtime, injects the buyer’s decision and speech back into the conversation, manages context

length, and enforces failure attribution. It instantiates one buyer policy per episode and injects it into the offer tool via hidden arguments (the seller’s tool schema never exposes the runtime or the buyer handle).

3.3 Leads, archetypes, and pricing

Leads are sampled from three demographic segments (younger/middle/older, with correlated income, household size, and risk distributions). Each lead draws a *temperature* (cold, lukewarm, warm, hot; near-uniform) that shifts trust and call tolerance, and one of ten *buyer archetypes* (Analytical, Relationship, Skeptic, Budget Hawk, Delegator, Procrastinator, Status Seeker, Protector, Comparison Shopper, Impulse Decider; uniform). The archetype determines which of seven sales-quality criteria (discovery, tailoring, objection handling, pressure calibration, value articulation, rapport, pacing) the buyer weights as primary, secondary, or de-emphasized when it evaluates an offer. A lead’s monthly insurance budget is a fraction of its income drawn from a wide range, producing an even spread of deal sizes (budgets span roughly \$52–\$952/month in a representative 100-lead episode). The catalog prices four plan types (Term, Whole, Universal Life, Disability Income) with a deterministic premium that scales with coverage and is modulated by age, risk class, term, and smoking status, giving the agent a real coverage-to-budget fitting problem: it must pull a quote whose premium fits the budget before it can credibly propose.

3.4 Tools and the canonical workflow

The seller is given eleven tools across four namespaces: CRM (search_leads, get_lead, add_note, pipeline_summary), calling (start_call, propose_offer, end_call), products (list_plans, quote_plan), and calendar (schedule_callback, list_callbacks). A productive call follows the pattern: start a call, briefly discover the need, pull a quote that fits the budget, propose the offer, and end the call. Two invariants make the task honest. First, exactly one call may be active at a time. Second—an anti-hallucination rail—propose_offer is rejected as an invalid action unless the agent has already pulled a quote for that lead, so the agent cannot manufacture revenue from a fabricated price. Each tool costs simulated time (proposing an offer costs the most, four minutes), so the time budget forces the agent to triage: it cannot call every lead, and long conversations crowd out closes.

3.5 Reward

The reward is a revenue-first weighted composite computed at episode end:

$$R = 1.00 \cdot \frac{\text{MRR}}{\text{MRR}_{\max}} + 0.10 \cdot \frac{\text{conversions}}{N} + 0.30 \cdot \text{budget_util} + 0.02 \cdot \text{completion} - 0.30 \cdot \text{dnc_violations} - 0.05 \cdot (0.1 \cdot \text{invalid_actions}),$$

where $\text{MRR}_{\max}$ is the sum of lead budgets, budget_util rewards capturing a high share of each converted lead’s budget, and the penalties cover compliance and malformed actions (full definitions in §A.1). Revenue dominates by design; the shaping terms are deliberately small, because in an earlier iteration a larger completion bonus created a degenerate *floor* strategy under GRPO—the model maximized the cheap terms and stopped exploring conversions—and shrinking the weights broke the trap. All six weights are configurable, and the environment additionally logs ~35 zero-weight diagnostic metrics (§A.2), so both the reward and its telemetry can be studied as experimental variables.

3.6 Failure attribution

The orchestrator separates two failure classes so that scores reflect the agent’s decisions, not the buyer’s serving reliability—a prerequisite for fair comparison across models and across days. *Seller errors* (no active call, proposing without a quote, unknown plan type, non-positive premium, calling a do-not-call lead) raise a deterministic error, increment the invalid-action counter, and return a recoverable message. *Buyer-model failures* (timeouts, malformed JSON, transport errors) are caught and resolved to a benign rejection, and are

never counted against the seller. The offer tool is structured to make this clean—record offer (deterministic, seller-attributable) → call buyer (isolated) → apply decision (deterministic)—so a buyer outage cannot manufacture an invalid action.

3.7 Context management for long horizons

At full scale an episode spans hundreds of tool calls and the transcript outgrows the context window. SALESBENCH compresses *deterministically*: when the prompt reaches a configurable fraction (default 80%) of the maximum sequence length, the orchestrator replaces older messages with a compact summary rendered from runtime state (time, revenue, conversions, completed-call outcomes, active call, pipeline counts), keeping the system prompt, briefing, and most recent messages verbatim. The summary uses no extra model call and fires at most a few times per episode, keeping the key-value cache stable for training (algorithm in §A.3). This compression is load-bearing at 100 leads (§4).

4 Experiments

We use SALESBENCH both as a training signal and as an evaluation suite, to demonstrate the full loop the benchmark is designed to support.

4.1 Setup

Curriculum. We train a 2B-parameter base model (Qwen3.5-2B) with GRPO (Shao et al., 2024) over four difficulty stages of increasing scale: 2 leads / 1 hour, 4 leads / 2 hours, 8 leads / 4 hours, and 20 leads / 10 hours. Only the first stage trains from scratch (to acquire the search→call→quote→propose workflow); each subsequent stage warm-starts from the previous checkpoint. Training uses the DEFAULT buyer exclusively. The buyer model is a small hosted model held fixed throughout.

Evaluation. We evaluate at full scale—50 and 100 leads—which is $2.5\text{--}5\times$ larger than any training stage. Each evaluation cell is 128 episodes on the held-out evaluation split (disjoint seeds from training), one rollout each. Because each episode uses a distinct seed, the 128 episodes per cell are independent draws, and we treat per-cell quantities as means over this sample; the standard error of the mean is the sample standard deviation across episodes divided by $\sqrt{128}$. We run the trained agent and the untrained base against all four buyer variants (DEFAULT, SKEPTICAL, IMPULSIVE, ANALYTICAL); the latter three are never seen in training. We report composite reward, per-lead conversion rate (*conv/lead*), captured MRR fraction (*mrr-cap*), and the fraction of episodes that terminate cleanly (*done*).

4.2 Main results

Table 1 reports the evaluation matrix. The trained agent improves over the untrained base on every cell and every metric. Against the DEFAULT buyer it raises per-lead conversion from 1.3% to 26.8% at 50 leads and from 0.3% to 18.5% at 100 leads, and composite reward from negative (−0.04) to positive (+0.19–0.32). Expressed as a ratio, the mean conversion lift over the untrained base is $21\times$ at 50 leads and $50\times$ at 100 leads; we report these lifts because they are striking, but note that the ratio is inflated by a near-zero baseline (the untrained model converts almost nothing at scale) and is sensitive to baseline noise, which is why we foreground the absolute conversion rates and the signed composite reward throughout.

4.3 Learning dynamics

The training run reveals a clear two-phase structure that the benchmark is built to expose. The untrained base model fails the *interface* before it can even attempt the task. In a representative diagnostic of three episodes it issued 43 offer attempts with zero quote calls, never started a well-formed call, and mis-filled the argument schema—placing “ACCEPT” into the offer’s next_step field, as if the buyer’s decision were something the seller could set—yielding a single accidental conversion; the aggregate evaluation confirms the pattern,

Scale	Agent / buyer	reward	conv/lead	mrr-cap	done
50 leads	untrained, DEFAULT	−0.039	0.013	0.002	0.469
	trained, DEFAULT	0.316	0.268	0.276	0.922
	trained, SKEPTICAL	0.133	0.160	0.139	0.875
	trained, IMPULSIVE	0.561	0.430	0.451	0.906
	trained, ANALYTICAL	0.285	0.243	0.249	0.922
100 leads	untrained, DEFAULT	−0.040	0.003	0.000	0.531
	trained, DEFAULT	0.194	0.185	0.180	0.906
	trained, SKEPTICAL	0.017	0.079	0.054	0.906
	trained, IMPULSIVE	0.212	0.197	0.194	0.922
	trained, ANALYTICAL	0.117	0.133	0.122	0.906

Table 1: Evaluation matrix on SALESBENCH (128 episodes per cell, one rollout each). *conv/lead*: per-lead conversion rate; *mrr-cap*: captured fraction of maximum achievable MRR; *done*: fraction of episodes terminating cleanly. The trained agent is trained only against the DEFAULT buyer; SKEPTICAL, IMPULSIVE, and ANALYTICAL are unseen.

with the base model averaging far more invalid actions and converting almost nothing (Table 1). The first thing the agent learns (within roughly the first 50 optimization steps) is the tool *syntax*: schema errors vanish and the mean invalid-action count per episode falls from about 6 to about 0.5. The second, much slower thing it learns is the *skill*—closing deals—which requires honoring the quote-before-propose constraint, fitting coverage to budget, and, importantly, *triage*: recognizing when an objection is about information rather than price and when to abandon a lead rather than burn four-minute offers retrying it. The curriculum makes this skill transfer cheaply: only the first 2-lead stage does substantial work (about 200 steps to near its ceiling), and the subsequent 4-lead stage starts already at 95.7% of its own ceiling, with later stages following the same fast-transfer pattern. Per-dialog-turn closing rate rises from ~ 0.003 for the base to ~ 0.112 for the trained agent, an over $35\times$ improvement, and do-not-call violations remain near zero throughout—the compliance rail holds even as the agent is optimized hard for revenue.

Two further points. *Revenue, not just conversions*: a benchmark scoring only “did the buyer say yes” would reward a policy that closes many tiny deals; the revenue-first reward and budget.util term instead reward closes with high premium relative to budget. The captured-MRR column of Table 1 tracks this—for the trained agent it moves in lockstep with conversion rate, so it is not buying conversions with give-away pricing. *Compliance holds under optimization*: do-not-call violations stay near zero throughout training and evaluation, even as the agent is optimized hard for revenue.

4.4 Scaling and generalization

The advantage widens with horizon (Figure 3, left). As leads double from 50 to 100, the untrained agent’s DEFAULT conversion collapses $4.3\times$ ($1.3\% \rightarrow 0.3\%$) and only half its episodes terminate cleanly ($\text{done} = 0.53$); the trained agent degrades $\sim 1.8\times$ and stays $> 90\%$ clean at both scales. The mean conversion lift over the base thus grows from $21\times$ at 50 leads to $50\times$ at 100—the longer-horizon regime is more *discriminating*, not merely harder.

Generalization across unseen counterparties (Figure 3, right). Trained only against the DEFAULT buyer, the agent sustains 7.9–19.7% per-lead conversion against all four variants at 100 leads—evidence of a transferable policy rather than counterparty idiosyncrasy. We scope this carefully: the variants share one buyer base model and differ only in their decision-guideline prompt (§A.6), so this is transfer across decision *styles*, not buyer *models* (the latter is future work, §5). They also form a difficulty axis—IMPULSIVE easiest, SKEPTICAL hardest (the agent’s clearest failure mode, $\sim 5\times$ conversion drop 50 $\rightarrow$ 100 leads vs. $1.5\text{--}2.2\times$ for others), though even there it keeps a $26\times$ lift over the base.

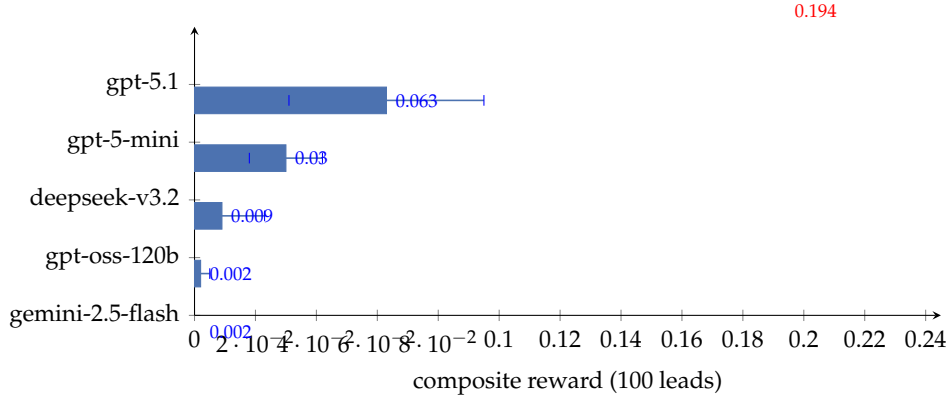


Figure 2: Frontier-model leaderboard on SALESBENCH at 100 leads (composite reward, DEFAULT buyer). The curriculum-trained 2B model (red) outranks every untrained frontier API model (blue) by roughly $3\times$; ranking is not a function of scale (the 120B gpt-oss-120b scores at the noise floor). Error bars are the standard deviation across the four episodes per model. Per-metric values in Table 2.

Model	reward	conv/lead	mrr-cap	done
Trained Qwen3.5-2B (ours)	0.194	0.185	0.180	0.906
gpt-5.1	0.063 ± 0.032	0.133	0.039	0.000
gpt-5-mini	0.030 ± 0.012	0.103	0.017	0.000
deepseek-v3.2	0.009 ± 0.014	0.060	0.010	0.000
gpt-oss-120b	0.002 ± 0.003	0.015	0.002	0.000
gemini-2.5-flash	0.002 ± 0.000	0.010	0.001	0.000

Table 2: Frontier-model leaderboard on SALESBENCH at 100 leads (DEFAULT buyer; untrained API models run through prime eval run, four episodes each, $\pm$ standard deviation across episodes; *done*: fraction of episodes terminating cleanly). The curriculum-trained 2B model (top row, from Table 1) outranks every untrained frontier model by roughly $3\times$, and is the only entry that processes the pipeline within budget. Ranking is not a function of scale: the 120B gpt-oss-120b scores at the noise floor. A few additional models were excluded due to inference-endpoint instability at this episode length (timeouts and routing errors during the long 100-lead rollouts).

4.5 A frontier-model leaderboard: capability, not scale

To check that SALESBENCH discriminates among strong models—and is not merely a tool-syntax check any capable model passes—we ran a leaderboard of frontier API models at the canonical 100-lead scale through the public evaluation path (prime eval run; same buyer and protocol, four episodes per model). Figure 2 and Table 2 report the results, with the curriculum-trained 2B model as a reference. Three findings stand out. The ranking is *not* a function of scale: the 120B gpt-oss-120b and gemini-2.5-flash sit at the noise floor (≈ 0.002), below deepseek-v3.2 and well below the much smaller gpt-5-mini. Most striking, the trained 2B model *tops the leaderboard* at 0.194, $\sim 3\times$ the best untrained frontier model (gpt-5.1, 0.063): targeted RL buys more than scale or general capability. And no untrained API model terminates cleanly at 100 leads (done = 0)—they exhaust budget or context before processing the pipeline, the triage failure the trained model overcomes (91% clean termination, Table 1). Small per-cell samples ($n=4$) are reported with the standard deviation across episodes.

4.6 Context compression is load-bearing

At 100 leads the trained agent triggers deterministic context compression (§3.7) ~ 8.5 times per episode. Disabling it is catastrophic: 47% of episodes error out on context overflow before the time budget is spent, versus $< 2\%$ with compression. Overflow caps achievable revenue and corrupts the completion signal, so without compression the benchmark would measure serving limits, not policy. State-derived summarization is therefore a requirement, not a convenience, for evaluating long-horizon agents under realistic context, and we ship it as a first-class part of the environment.

5 Discussion

What SALESBENCH measures that single-trajectory benchmarks do not. Three things, by construction: a *policy property* (the world is fixed, so a score is repeatable across runs and comparable across models on the same seeds); *allocation* (revenue under a hard budget rewards triaging leads, fitting coverage to budget, and stopping—behaviors a binary success metric cannot express); and *influence under uncertainty* (the grader is an adaptive, sometimes-uncooperative agent, and the buyer-variant axis turns “did it overfit the counterparty?” into a quantitative test).

Reward design is fragile under RL. Almost any shaping term becomes a *floor trap* once a model can collect it without selling—a concrete instance of the reward-misspecification failures of Pan et al. (2022) and Skalse et al. (2022). An over-weighted completion bonus taught the agent to end episodes rather than close deals; a conversion-plus-completion conjunction gave GRPO a cheap floor to lock onto while ceasing to explore. The fix each time was to shrink the shaping weights until the only large gradient pointed at revenue. Because the reward is fully exposed and configurable (§3.5), SALESBENCH doubles as a testbed for reward-hacking and shaping robustness in long-horizon RL.

The pattern is portable. None of the design—three-party orchestration, deterministic reproducible world, seller/counterparty failure attribution, buyer-variant axis—is specific to insurance. The recipe: use a language model as the counterparty, keep the reward grounded in environment state rather than a judge, and evaluate on long-horizon operational outcomes under a hard budget. It applies to procurement, support-deflection, tutoring, and scheduling—any domain where an agent runs a stateful workflow under a budget while influencing an agent it cannot control. A natural extension is to evaluate deployed agent *products* (with their own scaffolding) rather than only raw API models, by exposing the SALESBENCH tools through the harness those products already speak.

Limitations. The buyer is a single small hosted model with four prompt variants; richer counterparty distributions (multiple base models, learned buyers, human-in-the-loop) would strengthen the generalization claim. Our training demonstration uses one 2B model and one RL algorithm; SALESBENCH is intended to be run across the model frontier, and the per-model leaderboard is the natural next step that the released harness supports directly. The simulator’s economics are stylized; while the *structure* of budgeting and counterparty influence is faithful, the price and persona distributions are synthetic and should not be read as a model of real insurance markets.

6 Conclusion

SALESBENCH is a long-horizon, agent-to-agent benchmark in which a seller agent maximizes revenue under a hard time budget while influencing a personality-driven buyer it cannot control. Its design—a reproducible world with stochastic agents, explicit seller/buyer failure attribution, and a built-in buyer-variant generalization axis—targets properties single-trajectory benchmarks omit. A 2B model trained on a four-stage curriculum lifts per-lead conversion from near zero to 18.5% at 100 leads, an advantage that *widens* with horizon, transfers to three unseen buyer variants, and tops a leaderboard of frontier API models by $\sim 3\times$. We release the environment, curriculum, and harness so the community

can place models on the benchmark, study reward and counterparty design as variables, and port the pattern to other budgeted, agent-to-agent domains.

Reproducibility statement

The environment is a deterministic state machine seeded per episode; identical seeds reproduce the lead population, prices, time costs, and state transitions exactly, so the only stochasticity is the seller and buyer models. We report the exact evaluation protocol (128 episodes per cell, one rollout each, held-out evaluation seeds disjoint from training), the curriculum stages, the reward weights (§3.5), and the context-compression policy (§3.7). The environment, curriculum configurations, buyer-variant prompts, and evaluation harness are released open-source.

Ethics statement

SALESBENCH simulates a sales domain in which an agent is optimized to persuade a counterparty toward a purchase. We emphasize that the buyer is a synthetic language model, not a person, and that the benchmark is intended to *measure and stress-test* agent behavior—including compliance, since a do-not-call violation is explicitly penalized—rather than to deploy persuasive agents against real consumers. Optimizing agents for persuasion carries dual-use risk; we believe an open, measurable benchmark that makes compliance and pressure-calibration first-class, scored quantities is a net positive for studying and constraining such behavior. The personas are synthetic and do not target real individuals or protected groups.

References

- Axel Backlund and Lukas Petersson. Vending-bench: A benchmark for long-term coherence of autonomous agents. *arXiv preprint arXiv:2502.15840*, 2025. URL <https://arxiv.org/abs/2502.15840>.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -Bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025. URL <https://arxiv.org/abs/2506.07982>.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning (ICML)*, 2009. URL <https://dl.acm.org/doi/10.1145/1553374.1553380>.
- William Brown. Verifiers: Environments for LLM reinforcement learning. <https://github.com/PrimeIntellect-ai/verifiers>, 2025. Open-source framework.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024. URL <https://arxiv.org/abs/2411.15124>.
- Mike Lewis, Denis Yarats, Yann N. Dauphin, Devi Parikh, and Dhruv Batra. Deal or no deal? end-to-end learning for negotiation dialogues. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017. URL <https://arxiv.org/abs/1706.05125>.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.03688>.

- Grégoire Mialon, Clémentine Fourier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2311.12983>.
- Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2201.03544>.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2305.15334>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2307.16789>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2209.13085>.
- Xuwei Wang, Weiyan Shi, Richard Kim, Yoojung Oh, Sijia Yang, Jingwen Zhang, and Zhou Yu. Persuasion for good: Towards a personalized persuasive dialogue system for social good. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. URL <https://arxiv.org/abs/1906.06725>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024. URL <https://arxiv.org/abs/2406.12045>.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2306.05685>.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2307.13854>.

A Appendix

The appendix documents the harness, reward, and prompts in enough detail to reproduce SALESBENCH from the released code. §A.1–A.2 specify the reward and logged metrics; §A.3 the orchestration loop, hidden-argument injection, and deterministic context-summary algorithm; §A.4–A.5 the tools, leads, and pricing model; §A.8–A.9 the verbatim seller and buyer prompts; §A.10 the curriculum and training setup; and §A.13 an additional live-evaluation artifact.

A.1 Reward specification

The episode reward is the weighted sum $R = \sum_k w_k r_k$ over the six terms below, computed once at episode end from runtime state. Let N be the lead count, $\mathcal{C}$ the set of converted leads, p_i the accepted premium and b_i the monthly budget of lead i , and $\text{MRR} = \sum_{i \in \mathcal{C}} p_i$.

- 463 • **Revenue** ($w=1.00$): $\text{MRR}/\text{MRR}_{\max}$, where $\text{MRR}_{\max} = \sum_{i=1}^N b_i$.
- 464 • **Conversion rate** ($w=0.10$): $|\mathcal{C}|/N$.
- 465 • **Budget utilization** ($w=0.30$): $\frac{1}{N} \sum_{i \in \mathcal{C}} \min(1, p_i/b_i)$.
- 466 • **Completion** ($w=0.02$): 1.0 if the pipeline was exhausted, 0.5 if the time budget was
467 exhausted, 0 if the invalid-action cap was hit or the episode is unfinished.
- 468 • **DNC penalty** ($w= -0.30$): number of do-not-call violations.
- 469 • **Invalid-action penalty** ($w= -0.05$): $0.1 \times$ the invalid-action count (i.e. effectively
470 -0.005 per invalid action).

471 All six weights are constructor arguments and can be swept as experimental variables.
472 The small completion and conversion weights are deliberate: a larger completion bonus
473 previously induced a floor strategy under GRPO (§5).

474 A.2 Logged diagnostic metrics

475 Beyond the reward terms, the environment logs ~ 35 zero-weight metrics for offline analysis:
476 raw revenue, conversions, calls started/completed, offers proposed/accepted/rejected,
477 hang-ups, do-not-call violations, time utilization, leads contacted, callbacks sched-
478 uled/completed, per-tool call counts, the number of context-summarization events, buyer-
479 model call/latency/timeout/slow-call counters, and a structured error-type code (decoding
480 OpenAI- and vLLM-style API error bodies to integer classes) used to diagnose serving
481 failures remotely. None affect the reward; all appear in rollout results.

482 A.3 Orchestration, hidden arguments, and context summarization

483 SALESBENCH is implemented as a stateful tool-environment on the VERIFIERS frame-
484 work (Brown, 2025). Each rollout owns a runtime and a freshly-instantiated buyer policy,
485 both stored in rollout state. Tool functions declare hidden parameters (`runtime`, `messages`,
486 `buyer_llm`) that are stripped from the schema the seller sees and injected by the orchestrator
487 at call time, so the seller’s tool interface exposes only domain arguments while the orches-
488 trator wires in per-rollout state. The seller’s turn loop is: emit tool call(s) (and optional
489 speech) $\rightarrow$ orchestrator validates and applies each call to the runtime $\rightarrow$ for `propose_offer`,
490 the orchestrator records the offer, queries the buyer policy, and applies the decision $\rightarrow$
491 buyer speech (a conversational reply, or the decision’s spoken reason) is appended as a user
492 message. The episode-completion stop condition fires when the runtime reports done.

493 *Context summarization.* Before each model turn the orchestrator checks the previous turn’s
494 prompt-token count; when it reaches τ (default 0.80) of the maximum sequence length, it
495 rebuilds the message list as [system prompt, briefing] + [state-derived summary] + [last k
496 messages] (default $k=10$), where the summary is rendered deterministically from runtime
497 state (elapsed/remaining minutes, revenue, conversions, per-call outcomes, active call,
498 pipeline counts). If the first kept message is a user turn, the summary is merged into it
499 to avoid consecutive user messages. No model call is involved, and the trigger typically
500 fires only a handful of times per 100-lead episode, keeping the key-value cache stable for
501 training.

502 A.4 Tools

503 The seller has eleven tools: `crm_search_leads`, `crm_get_lead`, `crm_add_note`,
504 `crm_pipeline_summary`, `calling_start_call`, `calling_propose_offer`, `calling_end_call`,
505 `products_list_plans`, `products_quote_plan`, `calendar_schedule_callback`, and
506 `calendar_list_callbacks`. Time costs (minutes): `search` 1, `start-call` 1, `quote` 1, `pro-`
507 `pose` 4, `end-call` 1, `schedule_callback` 1, conversational message 2; others 0. Invariants: at
508 most one active call at a time; `propose_offer` requires a prior `quote_plan` for the same lead
509 (anti-hallucination rail); a lead becomes exhausted after its per-lead call cap.

A.5 Leads and pricing model

Leads are drawn from three demographic segments with correlated age/income/household/risk distributions, then assigned a temperature (cold/lukewarm/warm/hot, shifting trust and call tolerance) and one of ten archetypes (uniform). Each lead’s monthly budget is $b = (\text{income}/12) \cdot u$ with $u \sim \mathcal{U}(0.008, 0.040)$, floored at \$25. The premium for a quote is closed-form:

$$\text{premium} = \frac{\text{coverage}}{1000} \cdot \text{base_rate}(\text{plan, age-band}) \cdot \text{risk_mult}(\text{risk class}) \cdot \text{term_mult} \cdot s,$$

where $\text{risk_mult} \in \{0.85, 1.00, 1.35\}$ for preferred/standard/substandard, term_mult applies to Term plans (0.75–1.28 for 10–30-year terms), and $s=0.82$ for non-smokers (else 1.0). Base rates increase with age band and differ by plan type (Term cheapest, Whole most expensive at matched coverage).

A.6 Buyer-prompt variants

All four buyer variants receive identical persona information (demographics, budget, latent need, trust, price sensitivity, archetype, temperature) and differ only in a “decision guidelines” block, isolating decision *style* from information. DEFAULT behaves realistically given the persona. SKEPTICAL defaults to rejection and demands clear, well-above-margin fit, targeting a ~25–30% acceptance rate. IMPULSIVE accepts readily once basic affordability and plan-fit floors are met but enforces hard floors against unaffordable or clearly mismatched offers. ANALYTICAL ignores rapport entirely and decides purely on affordability ratio, coverage-to-income fit, and plan-type match. The variants are intended as an ablation axis for counterparty generalization, not as a claim about real buyer populations.

A.7 Episode termination and metrics

An episode terminates for one of three reasons: the time budget is exhausted, the pipeline is exhausted (all leads converted, on the do-not-call list, or out of call attempts), or the invalid-action cap is reached. The shaped completion bonus rewards these in order of quality (pipeline exhaustion > time exhaustion > invalid-action cap). Beyond the reward terms, the environment logs raw revenue, conversions, calls started and completed, offers proposed/accepted/rejected, hang-ups, do-not-call violations, time utilization, callbacks, buyer-model latency/timeout/slow-call counters, and a structured error-type code, all at zero weight, for offline analysis.

A.8 Seller system prompt

The seller receives the following system prompt (verbatim), followed by a per-episode briefing stating the lead count and time budget.

You are a sales agent operating a structured insurance pipeline.
 Core objective: Maximize converted monthly premium by closing deals with calling_propose_offer. Revenue is ONLY earned when a buyer ACCEPTs an offer; follow-ups, emails, and callbacks generate zero revenue.
 Critical workflow --- every call must follow this pattern: (1) calling_start_call; (2) one brief message to greet and discover needs; (3) products.quote_plan --- one quote that fits the lead’s budget; (4) calling_propose_offer --- the only way to convert; (5) if REJECT, try one revised offer, then end; (6) calling_end_call.
 Execution rules: propose on every call before ending; keep conversations short (the buyer decides on the offer tool, not prose); one quote per lead; do not promise emails/PDFs (they do not exist); do not hallucinate prices (quote first, pass the exact premium); one active call at a time; avoid do-not-call violations and invalid actions.
 Strategy: prioritize warm/hot leads with high need and adequate budget; match coverage to budget (premium-to-budget ratio 0.5--1.0 is ideal); more

558 calls with proposals beats fewer calls with long conversations; be concise.
 559 Tool time costs (min): search=1, start_call=1, quote=1, propose=4,
 560 end_call=1, schedule_cb=1. Others=0.

561 **A.9 Buyer decision prompt**

562 On each proposed offer the buyer policy is prompted with a system message assembled from
 563 the lead’s persona and one of the four decision-guideline blocks (§A.6), then asked to return
 564 JSON {decision, reason, request_dnc}. The persona section lists name, age, income,
 565 household and dependents, risk class, monthly budget, need/trust/price-sensitivity scores,
 566 contact count and tolerance, temperature, and archetype; a personality paragraph from the
 567 archetype; an “how to evaluate the agent’s approach” section ranking the seven sales-quality
 568 criteria as primary/secondary/de-emphasized per the archetype; and a temperature-specific
 569 readiness paragraph. During conversation (not offer evaluation) a lighter prompt asks the
 570 buyer to reply in character in 1–3 sentences without making a purchase decision. Buyer
 571 calls run on a shared thread pool with a hard timeout; on any failure the offer resolves to a
 572 benign rejection (§3.6).

573 **A.10 Curriculum and training setup**

574 We train Qwen3.5-2B with GRPO (Shao et al., 2024) over four difficulty stages—2 leads/1h,
 575 4/2h, 8/4h, 20/10h—warm-starting each stage from the previous checkpoint; only the first
 576 trains from scratch. Each stage uses generated episodes on the train split (seeds disjoint from
 577 eval via fixed split offsets), 32 rollouts per example for group-relative advantages, online
 578 difficulty filtering, and the DEFAULT buyer throughout. Sampling temperature is 1.0. The
 579 first stage does the bulk of the work (~200 steps to near its ceiling); later stages converge
 580 quickly (the 4-lead stage begins at 95.7% of its ceiling). Evaluation uses the held-out eval
 581 split at 50 and 100 leads, 128 episodes per cell, one rollout each.

582 **A.11 Determinism and pricing checks**

583 We verify that two runtimes constructed from the same seed produce identical leads (identi-
 584 fiers, names, ages, incomes, latent needs), that the maximum achievable MRR equals the
 585 exact sum of lead budgets, that premiums are monotone in risk class (preferred < standard
 586 < substandard) and ordered across plan types (term < universal life < whole at matched
 587 coverage), and that the non-smoker discount is exactly the specified multiplier. These checks
 588 are part of the released test suite.

589 **A.12 Scaling and generalization figure**

590 **A.13 Live API-model evaluation**

591 As an independent check of the released harness—and a verified counterpart to the pre-
 592 liminary sweep of Table 2—we ran two frontier API models through the public evaluation
 593 path (prime eval run) end-to-end, with the buyer served by gpt-5-mini through the same
 594 inference endpoint. Table 3 reports the results. Three findings reproduce the paper’s central
 595 claims directly. First, the benchmark sharply separates models that pass the agentic inter-
 596 face from those that do not: gpt-5-mini drives the full search→call→quote→propose loop
 597 with near-zero invalid actions and converts 25–27% of leads, whereas gpt-oss-120b—an
 598 order of magnitude larger—scores at the noise floor (composite reward ≈ 0), failing or
 599 erroring on most episodes. Capability on SALESBENCH, in other words, is not a function
 600 of scale. Second, gpt-5-mini’s per-lead conversion at 50 leads (0.250) lands within noise
 601 of the curriculum-trained 2B model’s 50-lead figure (Table 1, 0.268), an independent cross-
 602 check that the trained agent is operating at a strong-API-model level of competence rather
 603 than merely beating a broken baseline. Third, do-not-call violations are zero for every
 604 cell, confirming the compliance rail is respected without special prompting. These are
 605 small-sample cells (reported with the standard deviation across episodes) and are meant as

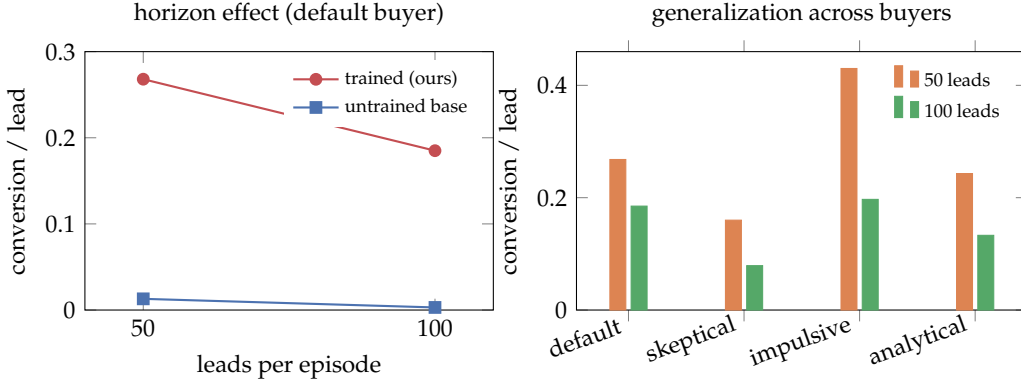


Figure 3: Left: the trained agent’s per-lead conversion degrades gently as the horizon doubles (50 $\rightarrow$ 100 leads) while the untrained base stays pinned near zero, so the relative advantage *widens* with horizon. Right: the trained agent (trained only on the DEFAULT buyer) sustains conversion across all four buyer-prompt variants at both scales, with SKEPTICAL the hardest. All values from the held-out evaluation (128 episodes per cell). This figure expands on §4.4.

Model	Scale	reward	conv/lead	mrr-cap	invalid
gpt-5-mini	20 leads ($n=3$)	0.158 ± 0.118	0.267	0.098	0.67
gpt-oss-120b	20 leads ($n=3$)	-0.001 ± 0.015	0.033	0.001	1.33
gpt-5-mini	50 leads ($n=8$)	0.114 ± 0.074	0.250	0.064	0.38

Table 3: Verified live evaluation through the released harness (prime eval run; buyer gpt-5-mini, DEFAULT variant; one rollout per episode; $\pm$ is the standard deviation across episodes). *conv/lead*: per-lead conversion rate; *mrr-cap*: captured fraction of maximum achievable MRR; *invalid*: mean invalid actions per episode. gpt-5-mini handles the agentic loop (25–27% conversion across 20- and 50-lead scales) while the much larger gpt-oss-120b scores at the noise floor, reproducing the “capability, not scale” finding. Notably gpt-5-mini’s 50-lead conversion (0.250) closely matches the curriculum-trained 2B model’s 50-lead figure in Table 1 (0.268), an external cross-check on the trained agent’s quality. All cells have zero do-not-call violations.

606 a reproducibility artifact rather than a final leaderboard; the released harness produces such
 607 cells directly via `tools/run_eval_matrix.sh`.